Co-funded by the European Union

Co-funded by the European Union. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Education and Culture Executive Agency (EACEA). Neither the European Union nor EACEA can be held responsible for them.

2023-1-EL01-KA220-HED-000166184

Final Project, Option 3: How much structure can you find without the labels?

An agricultural co-op receives unlabelled deliveries of mixed wheat grain and wants to know how many distinct varieties are present, and how cleanly they separate, before committing to any sorting process. In this project you recover that structure using clustering alone.

In this project you take a data set with a known grouping, hide that grouping from yourself, and see how much of it you can recover using clustering alone. The labels are kept aside until the very end, where they serve only as an answer key.

Central question: how much of the real class structure can you recover without ever using the labels?

The data set is Seeds: 210 wheat kernels described by seven geometric measurements such as area, perimeter, and kernel length. You are given the measurements to work with, and the variety labels are set aside for the final check.

This project draws on Module 4. There we scaled features, reduced them with PCA, clustered with K-Means, chose the number of clusters using the elbow and silhouette methods, and validated the result against a known label with a cross-tabulation. Here you carry out that whole process yourself and judge how faithfully the clusters match the real varieties.

How this notebook is organised. The cells in Part A are already written for you. They handle the import, the download, and the scaling. From Part B onward, the cells are yours to complete. Each one carries comments describing what to do, along with a few hints, but not the finished code. Work through them in order, and resist looking at the varieties until Part D.

Part A. Setup and data

We start with the imports. This cell loads everything the notebook needs, so we run it once at the top.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_openml
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

Loading the data

We download the Seeds data set. The seven measurements go into `X`, and the true varieties go into `y_true`. We put the varieties aside now and do not touch them until Part D, so that the clustering is done with no knowledge of the answer.

In [2]:
```
# The Seeds data set: 210 kernels described by seven geometric measurements.
# It is small and fully numeric,
# which suits clustering well.
feature_names = ["area", "perimeter", "compactness", "kernel_length",
                 "kernel_width", "asymmetry", "groove_length"]

seeds = fetch_openml(name="seeds", version=1, as_frame=True)

X = seeds.data.copy()
X.columns = feature_names          # give the seven measurements readable names

# The true varieties. We set these aside now and do NOT use them until Part D,
# where they serve only as an external check on the clustering.
y_true = seeds.target.to_numpy().astype(int)

print("Rows and features:", X.shape)
X.head()
```

```
Rows and features: (210, 7)
```

Out[2]:

	area	perimeter	compactness	kernel_length	kernel_width	asymmetry	groove_length
0	15.26	14.84	0.8710	5.763	3.312	2.221	5.220
1	14.88	14.57	0.8811	5.554	3.333	1.018	4.956
2	14.29	14.09	0.9050	5.291	3.337	2.699	4.825
3	13.84	13.94	0.8955	5.324	3.379	2.259	4.805
4	16.14	14.99	0.9034	5.658	3.562	1.355	5.175

Scaling the features

Clustering relies on distances between points, so a feature measured on a larger scale would count for more than the others. We put every feature on the same footing with `StandardScaler`, as in Module 4.

In [3]:
```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("Scaled feature matrix:", X_scaled.shape)
```

```
Scaled feature matrix: (210, 7)
```

Part B. Explore, without labels

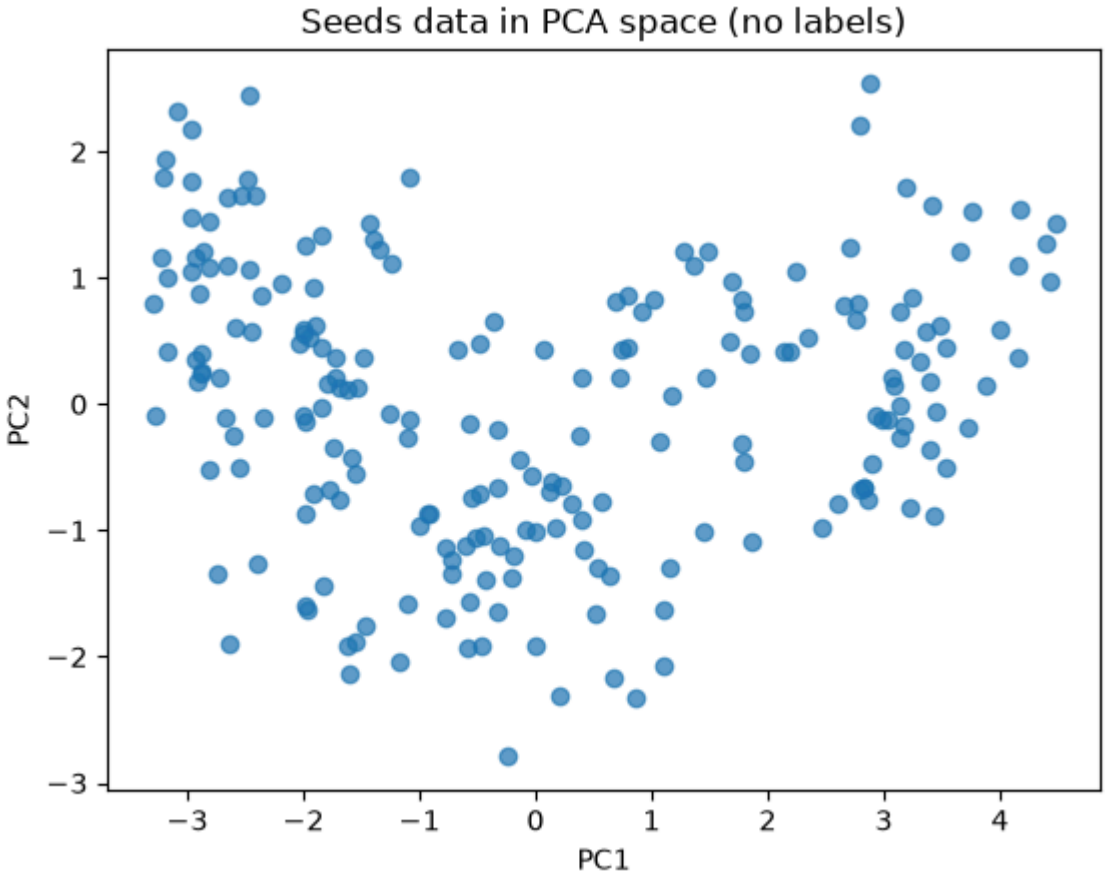
From here on, the cells are yours to complete.

First, look at the shape of the data. Seven measurements are too many to plot at once, so reduce them to two with PCA and draw a scatter. This is the same reduction you used in Module 4.

In [4]:
```
# Reduce the seven features to two so the data can be drawn on a flat plot.
#   - use PCA(n_components=2) and fit_transform on X_scaled, keeping the result
as X2
#   - draw a scatter of X2 using a SINGLE colour: at this stage we study the
shape
#     of the data without using any labels
#   - print pca.explained_variance_ratio_ to see how much of the variation the
#     two components capture
pca = PCA(n_components=2, random_state=42)
X2 = pca.fit_transform(X_scaled)
print(X2.shape)
print(pca.explained_variance_ratio_)
print(pca.explained_variance_ratio_.sum())

plt.scatter(X2[:, 0], X2[:, 1], alpha=0.7)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("Seeds data in PCA space (no labels)")
plt.savefig("../figures/1_pca_scatter_unlabelled.png", dpi=150,
bbox_inches="tight")
plt.show()
```

```
(210, 2)
[0.71874303 0.17108184]
0.8898248618491238
```



What do you see?

Write one or two sentences here describing the scatter. How many groups seem to be present? Do any of them run into one another?

From small clustered points and general direction changes it can be seen that there are 3 groups with some overlap between the middle group and the edge groups. The two principal components keep 89% of the variation and therefore the above figure is a good representation of the seven measurements.

Part C. Cluster and choose the number of groups

Now decide how many clusters the data supports, using the two methods from Module 4. The elbow method looks at inertia, the total spread of points around their cluster centres, and finds where adding another cluster stops helping much. The silhouette score measures how well separated the clusters are, where higher is better.

In [5]:
```
# Work out a sensible number of clusters.
# For each k from 2 to 7:
#   - fit KMeans(n_clusters=k, n_init=10, random_state=42) on X_scaled
#   - record the inertia (km.inertia_) for the elbow method
#   - record the silhouette score, silhouette_score(X_scaled, km.labels_)
# Then plot inertia against k, and silhouette against k, as two small charts.
# For the elbow, look for where the inertia stops dropping sharply.
# For the silhouette, look for the highest value.
inertias = []
sils = []
```

```python
ks = range(2, 8)
for k in ks:
    km = KMeans(n_clusters=k, n_init=10, random_state=42)
    km.fit(X_scaled)
    inertias.append(km.inertia_)
    sils.append(silhouette_score(X_scaled, km.labels_))

print(pd.DataFrame({"k": ks, "inertia": inertias, "silhouette": sils}))

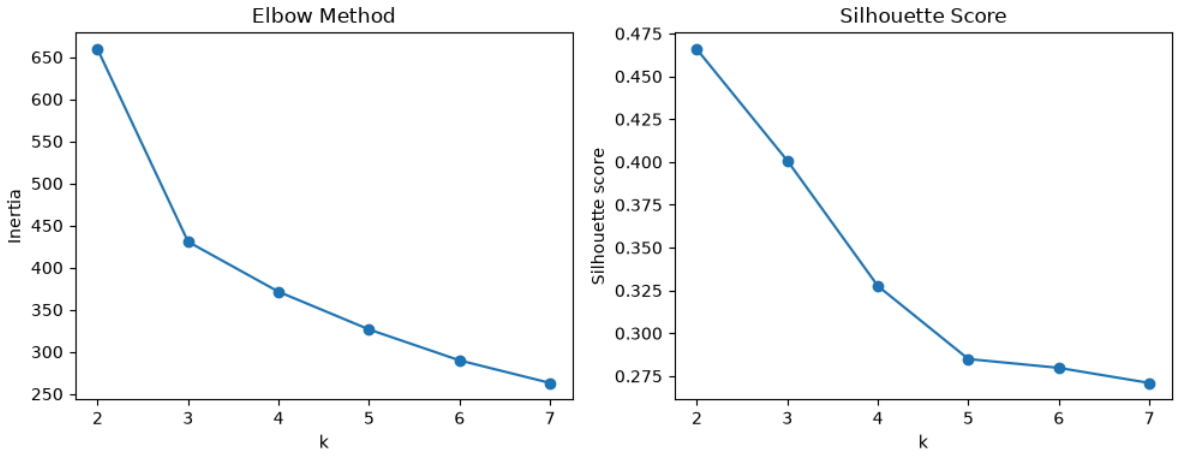
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.plot(list(ks), inertias, marker="o")
plt.xlabel("k")
plt.ylabel("Inertia")
plt.title("Elbow Method")

plt.subplot(1, 2, 2)
plt.plot(list(ks), sils, marker="o")
plt.xlabel("k")
plt.ylabel("Silhouette score")
plt.title("Silhouette Score")

plt.tight_layout()
plt.savefig("../figures/2_elbow_silhouette.png", dpi=150, bbox_inches="tight")
plt.show()
```

```
   k     inertia  silhouette
0  2  659.171754    0.465772
1  3  430.658973    0.400727
2  4  371.301721    0.327574
3  5  326.513695    0.284930
4  6  289.804037    0.279785
5  7  262.973618    0.270946
```



How many clusters?

The elbow and the silhouette will not always point to the same value. That disagreement is part of the exercise. **Write one or two sentences here** stating the value of k you will use and your reason, based on the two charts above.

Based on the two charts the number of clusters is clearly between 2 and 4. A closer analysis shows that the elbow is clearly at k=3 with inertia reduced significantly between two and three clusters. Considering the silhouette score it is decided to use k=3 as it is

not much lower than k=2. Despite the slight disagreement of the methods the elbow is the deciding factor. Taking into account the PCA plot it reinforces the choice of three clusters.

Fit K-Means

Fit K-Means at the number of clusters you decided on, and keep the cluster assignment for each kernel.

In [6]:
```
# Fit K-Means at your chosen number of clusters.
#   - set k to the value you argued for above
#   - fit KMeans(n_clusters=k, n_init=10, random_state=42) on X_scaled
#   - keep the cluster assignments as `labels`
k = 3

km = KMeans(n_clusters=k, n_init=10, random_state=42)
labels = km.fit_predict(X_scaled)
print(labels)
print(pd.Series(labels).value_counts())
```

```
[2 2 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2 2 0 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 0 0 0 2 0 2 2 2 2 2 0 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 2 1 1 2 1 2 2 1 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 2 0 2 0 2 0 0 0 0 0 0 0 0]
0    72
2    71
1    67
Name: count, dtype: int64
```

Part D. Judge against the labels

We now bring in the true varieties for the first time, only as an external check. The clustering itself is finished; nothing below changes it. We simply ask how well the groups you found line up with the real varieties, exactly as we validated the clusters in Module 4.

In [7]:
```
# Reveal the varieties only now, after the clustering is complete.
print("Varieties present:", np.unique(y_true))

# Cross-tabulate the cluster labels against the true varieties.
# Use pd.crosstab(labels, y_true). Each row is a cluster, each column a variety.
# A clean clustering has most of each row concentrated in a single column.
ct = pd.crosstab(labels, y_true, rownames=["cluster"], colnames=["variety"])
print(ct)
print(pd.crosstab(labels, y_true, rownames=["cluster"], colnames=["variety"],
normalize="index"))
```

```
Varieties present: [1 2 3]
variety   1   2   3
cluster
0         6   0  66
1         2  65   0
2        62   5   4
variety         1         2         3
cluster
0        0.083333  0.000000  0.916667
1        0.029851  0.970149  0.000000
2        0.873239  0.070423  0.056338
```

Measuring the agreement

Turn the table into a single number with a purity score. For each cluster, take the count of its most common variety; add those counts across all clusters; then divide by the total number of kernels. A value near 1 means each cluster is almost entirely one variety.

Purity tends to increase when more clusters are created, so interpret it together with your chosen value of `k` and the cross-tabulation rather than using it on its own.

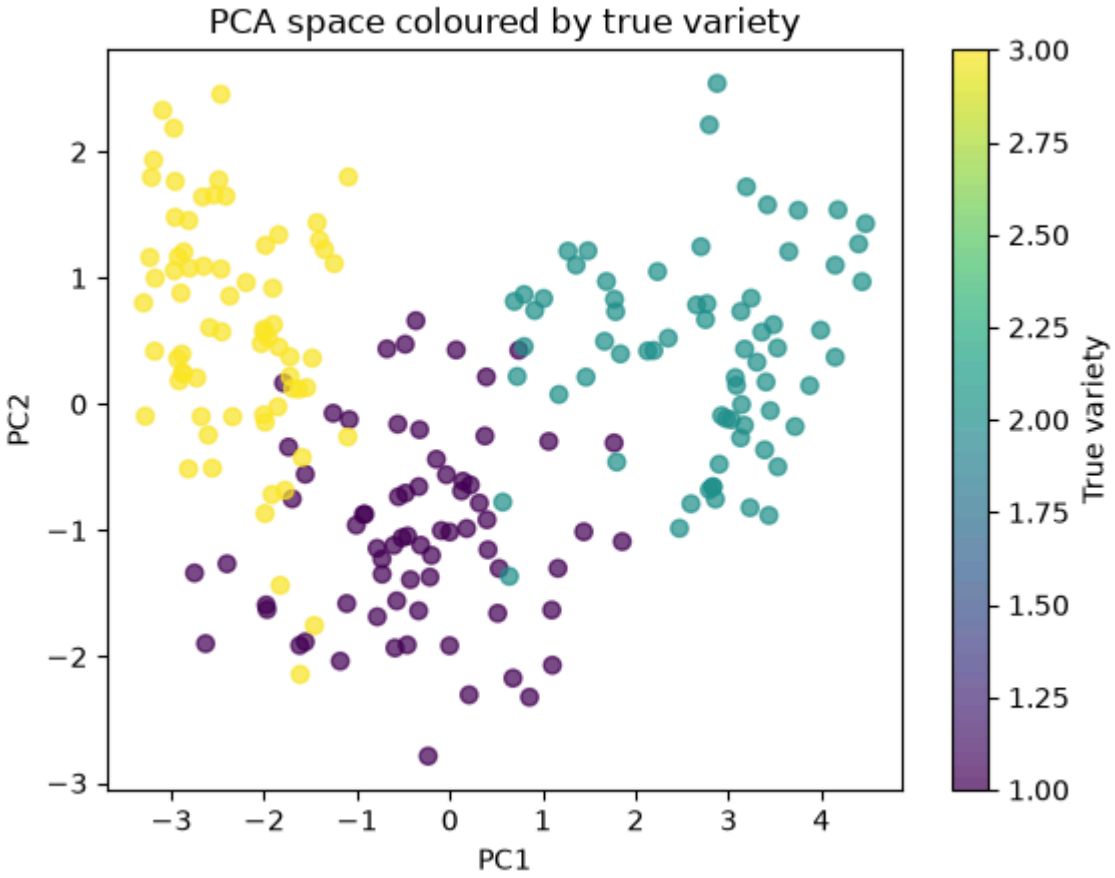
In [8]:
```python
# Compute the purity score from `ct`.
#   - for each cluster (each row), take the largest count: ct.max(axis=1)
#   - add these across clusters, then divide by the total, ct.values.sum()
purity = ct.max(axis=1).sum() / ct.values.sum()
print(ct.max(axis=1).sum(), ct.values.sum())
print(purity)
```

```
193 210
0.919047619047619
```

Back to the picture

Connect the numbers to the shape you saw earlier. Redraw the PCA scatter, but this time colour each point by its true variety. Interpret this picture cautiously: K-Means used all seven scaled features, while the plot shows only two PCA dimensions. Points that overlap here may still be separated by information in the other dimensions.

In [9]:
```python
# Redraw the scatter of X2 (from Part B), now coloured by the true variety
y_true.
# In plt.scatter(...), use c=y_true, cmap="viridis", and alpha=0.7.
# Add axis labels and a colour bar labelled "True variety".
plt.scatter(X2[:, 0], X2[:, 1], c=y_true, cmap="viridis", alpha=0.7)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA space coloured by true variety")
plt.colorbar(label="True variety")
plt.savefig("../figures/3_pca_true_varieties.png", dpi=150, bbox_inches="tight")
plt.show()
```



What each cluster captured

Finally, see what each cluster actually represents. The cluster centres are stored in scaled units, so turn them back into the original measurements with `scaler.inverse_transform`, and read across each row to picture the typical kernel in that group.

In [10]:
```
# Describe each cluster by its centre, in the original measurement units.
#   - the centres in scaled space are km.cluster_centers_ (from your fitted
model)
#   - convert them back with scaler.inverse_transform(...)
#   - put the result in a DataFrame with columns=feature_names so it reads
clearly
centers = scaler.inverse_transform(km.cluster_centers_)
profiles = pd.DataFrame(centers, columns=feature_names)
print(profiles)
```

```
        area  perimeter  compactness  kernel_length  kernel_width  asymmetry  \
0  11.856944  13.247778     0.848253       5.231750      2.849542   4.742389
1  18.495373  16.203433     0.884210       6.175687      3.697537   3.632373
2  14.437887  14.337746     0.881597       5.514577      3.259225   2.707341

   groove_length
0       5.101722
1       6.041701
2       5.120803
```

Ethical considerations

Clustering will always return groups, whether or not they reflect anything real. Treating those groups as fixed, meaningful categories, and acting on them, can do harm if the data does not truly support them.

Write two or three sentences here on the risks of over-reading this result. Consider what could go wrong if the co-op treated your clusters as definite varieties, and how the purity score and your choice of k should temper the confidence of any claim you make.

If the co-op treated the three clusters as definite varieties approximately 8% of the kernels would go in the wrong group. It is important to note that every error involved variety 1. The sorted lots would be mixed exactly where the varieties look most alike. If the choice of k was different, such as the silhouette-preferred k=2 purity would be reduced. Purity tends to go up with increasing number of clusters so purity alone could make a wrong number of groups look convincing. The level of claim that can be made based on the purity score is that about three varieties are plausibly present in data like this sample.

Wrapping up

Bring your findings together. **Write a short paragraph** that answers the central question directly:

- How many groups did the diagnostics support, and how did you decide when the elbow and the silhouette disagreed?
- From the cross-tabulation and the purity score, how much of the real variety structure did the clustering recover?
- Which variety was separated most cleanly, and which two were the hardest to tell apart? Does the coloured scatter support this interpretation?
- Why must you be cautious when judging a seven-dimensional clustering from a two-dimensional PCA plot?

What to hand in: this completed notebook, with the diagnostic charts, the cross-tabulation, the purity score, both scatter plots, and the cluster profiles visible in the output, and your written answers in the three markdown cells (Part B, Part C, and this one).

The diagnostics supported three groups. The elbow was clearly at k = 3 while the silhouette score was highest at k = 2 at 0.466 while for k = 3 it was 0.401 which pulled in the opposite direction of the elbow. In the end, k = 3 was selected because the elbow was sharp and the PCA scatter, which keeps 89% of the variation, showed three loose groups.

Each cluster was composed of largely one variety and in the end 193 of 210 (0.919 purity) kernels were correctly placed in a cluster led by their own variety. Therefore, clustering recovered most, even if not all, of the real structure without being based on the labels.

Variety 2 was separated most cleanly at 97% and it should be noted that there was no mixing between variety 2 and 3 in their clusters. Varieties 1 and 3 were the hardest pair to tell apart followed by varieties 1 and 2. The coloured scatter clearly supports this interpretation as variety 1 is located in the middle and overlapping with variety 3 on the

left and variety 2 on the right. Interpreting what each cluster captured it can be seen that variety 3 has small kernels, variety 1 medium, and variety 2 large.

A two-dimensional PCA plot is a compromise that is made to provide a representative visualisation of the data. The goal here was to capture most of the variation with two principal components and capturing 89% of it worked well. The rest of the variation can be the reason that points were separated in different clusters but shown to overlap on the coloured two-dimensional PCA plot. K-Means grouped the kernels using all seven measurements.